Dashboard Builder

Step-by-step guide to run the project on your computer

This app lets you build dashboards like in Canva or Figma. You add **sections** and put **text**, **images** and **charts** inside them, then drag, resize and save everything.

You don't need to know how the code works to follow this guide. Just do the steps in order.

What's inside this guide

Part 1: Install the needed software (one time only)

Part 2: Open the project folder in a terminal

Part 3: Way A, the easiest: run everything with Docker (about 5 minutes)

Part 4: Way B, the developer way: run with Node.js (if you want to change the code)

Part 5: How to use the app

Part 6: Stop, start again, reset

Part 7: Something went wrong? (problems and fixes)

Part 8: Quick command cheat sheet

Before you start: choose your way

There are two ways to run the project. **You only need ONE of them.**

RECOMMENDED

Way A: Docker (easiest)

One command starts everything: the database and the website.

- You only install **Docker Desktop**
- No database setup, no config files
- Opens at `http://localhost:5000`

FOR DEVELOPERS

Way B: Node.js

Use this if you want to **edit the code** and see changes instantly.

- Install **Docker Desktop** and **Node.js**
- A few more commands
- Opens at `http://localhost:5173`

> **Tip:** Not sure which one? Pick **Way A**.

Part 1: Install the needed software (one time only)

1

Install Docker Desktop (needed for both ways)

Download it from docker.com/products/docker-desktop and install it with the default options. On Windows, restart the computer if it asks you to.

After installing, **open Docker Desktop** and wait until it says **"Engine running"** (bottom-left corner). Docker Desktop must be **open** every time you use the project.

> **Tip:** On Windows, if Docker asks to install or update **WSL**, click yes and follow its instructions.

2

Install Node.js (only needed for Way B)

Download the **LTS** version (the green button) from nodejs.org and install it with the default options.

3 Check that everything is installed

Open a terminal (see Part 2) and type these commands. Each one should print a version number:

```
docker --version
docker compose version
node -v          (Way B only: should be v18 or higher)
npm -v           (Way B only)
```

> **Important:** If you get "command not found", close the terminal, open a new one and try again. If it still fails, restart the computer.

Part 2: Open the project folder in a terminal

1 Unzip the project

If you got the project as a `.zip` file, right-click it and choose **Extract All** (Windows) or double-click it (Mac). You'll get a folder with `backend`, `frontend`, `docker-compose.yml` and other files inside.

2 Open a terminal inside that folder

Windows	Open the project folder in File Explorer. Click the **address bar** at the top, type `cmd` and press **Enter**. A black window opens, already in the right folder.
Mac	Open **Finder**, right-click the project folder and choose **New Terminal at Folder**. (If you don't see it: System Settings → Keyboard → Keyboard Shortcuts → Services → turn on "New Terminal at Folder".)
VS Code	Open the folder in VS Code, then use the menu **Terminal** → **New Terminal**.

> **Important:** Every command in this guide must be typed in the terminal **inside the project folder**, the one that contains `docker-compose.yml`. To check, type `dir` (Windows) or `ls` (Mac) and look for that file.

Part 3: Way A, run everything with Docker (easiest)

1 Make sure Docker Desktop is open

It should say **"Engine running"**.

2 Start the project

In the terminal (inside the project folder), type:

```
docker compose up --build
```

The **first time takes 3 to 10 minutes**, because Docker downloads and builds everything. Lots of text will scroll by; that's normal.

> **Success check:** You are ready when you see this line: `Dashboard Builder running at http://localhost:5000`

> **Important: Leave this terminal window open.** Closing it stops the app.

3 Open the app in your browser

Go to: **http://localhost:5000**

Sign in with the demo account:

Email: **demo@example.com** Password: **Demo@1234**

You can also click **Create one** to make your own account.

4 Stop it when you are done

Click the terminal window and press **Ctrl + C**. Then type:

```
docker compose down
```

Your dashboards are **kept**. Next time, just open Docker Desktop and run `docker compose up` again (no `--build` needed).

That's it for Way A. Skip to **Part 5** to learn how to use the app.

Part 4: Way B, run with Node.js (developer way)

Use this way if you want to change the code. You need **Docker Desktop** (for the database) and **Node.js**.

1 Start the database

Make sure Docker Desktop is open, then run:

```
docker compose up -d mysql
```

This starts only the MySQL database in the background. Wait about 20 seconds the first time.

> **Tip:** Already have MySQL 8 installed on your computer? You can use it instead of Docker. Just put your own MySQL username and password in the `.env` file in step 3.

2 Install the project libraries

```
npm install
```

This takes 1 to 3 minutes and creates a `node_modules` folder.

> **Tip:** If you see yellow **"npm warn"** lines (for example about `esbuild` or install scripts), you can ignore them. Only red **"npm error"** lines are a problem.

3 Create the settings file (.env)

Copy the example settings file:

Windows

```
copy backend\.env.example backend\.env
```

Mac / Linux

```
cp backend/.env.example backend/.env
```

Now create a secret key for logins. Run this, then **copy the long text** it prints:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Open the settings file:

Windows

```
notepad backend\.env
```

Mac / Linux

```
open -e backend/.env
```

Find the line that starts with `JWT_SECRET=` and replace everything after the `=` with the text you copied. **Save** and close the file.

> **Tip:** The database settings in this file already match the Docker database (user `root`, password `rootpass`), so you don't need to change them.

4 Create the database tables and sample data

```
npm run db:setup
```

> **Success check:** You should see: `Database "dashboard_builder" is ready.`

5 Start the app

```
npm run dev
```

This starts two parts at once: the **backend** (API, port 5000) and the **frontend** (website, port 5173).

> **Success check:** Ready when you see `Dashboard Builder running at http://localhost:5000` and a line containing `http://localhost:5173/`.

Open **http://localhost:5173** in your browser and sign in:

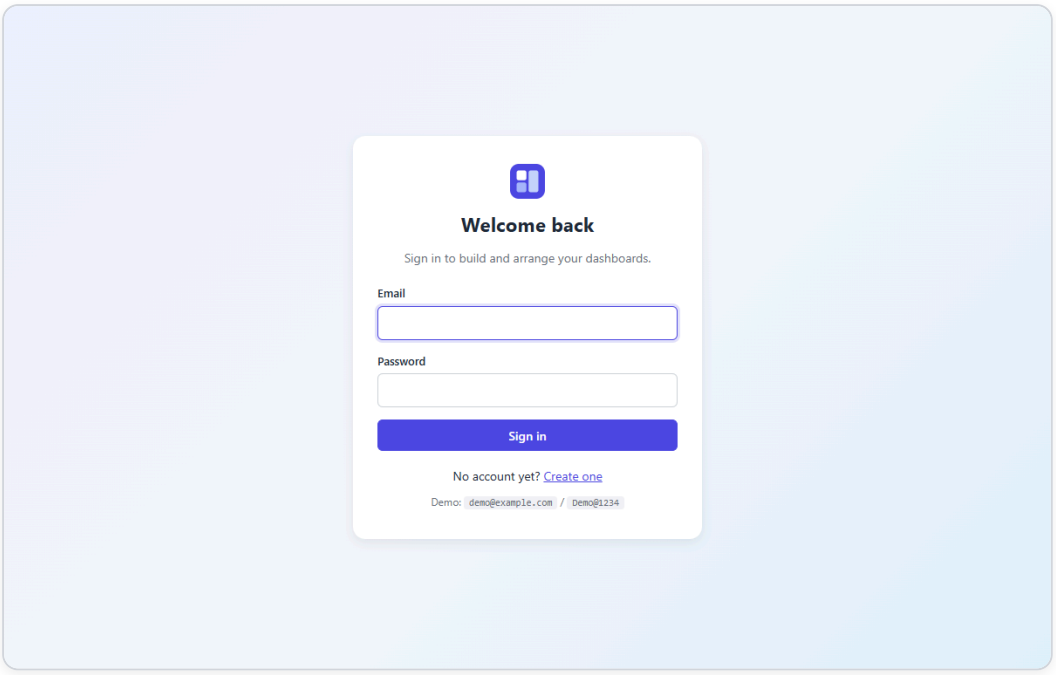
Email: **demo@example.com** Password: **Demo@1234**

> **Important:** In Way B always open port **5173**, not 5000. When you edit files in `frontend/src` the page updates automatically, and the backend restarts by itself when you edit `backend/src`.

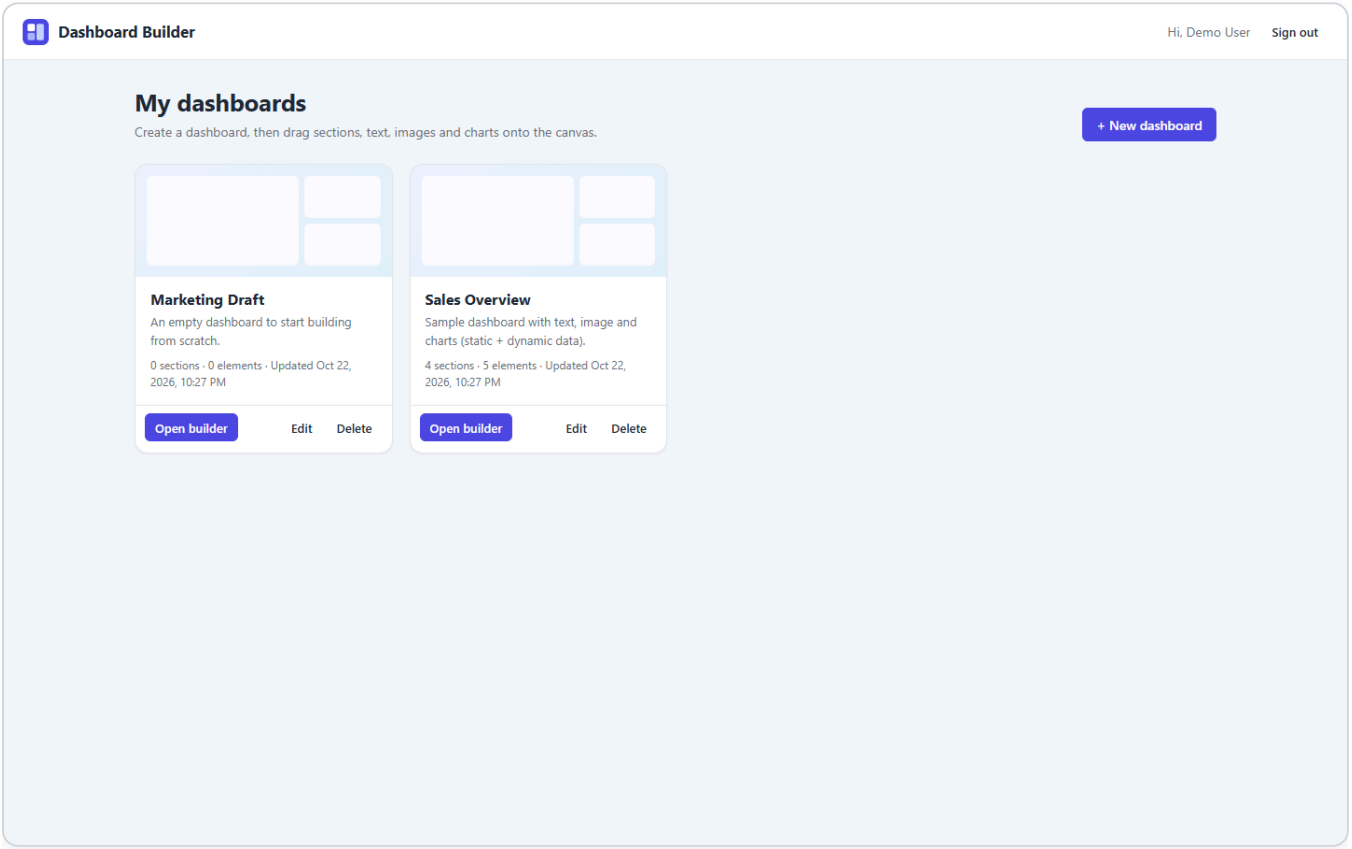
6 Stop it

Press **Ctrl + C** in the terminal. Next time you only need: open Docker Desktop, run `docker compose up -d mysql`, then `npm run dev`. Steps 2 to 4 are one-time only.

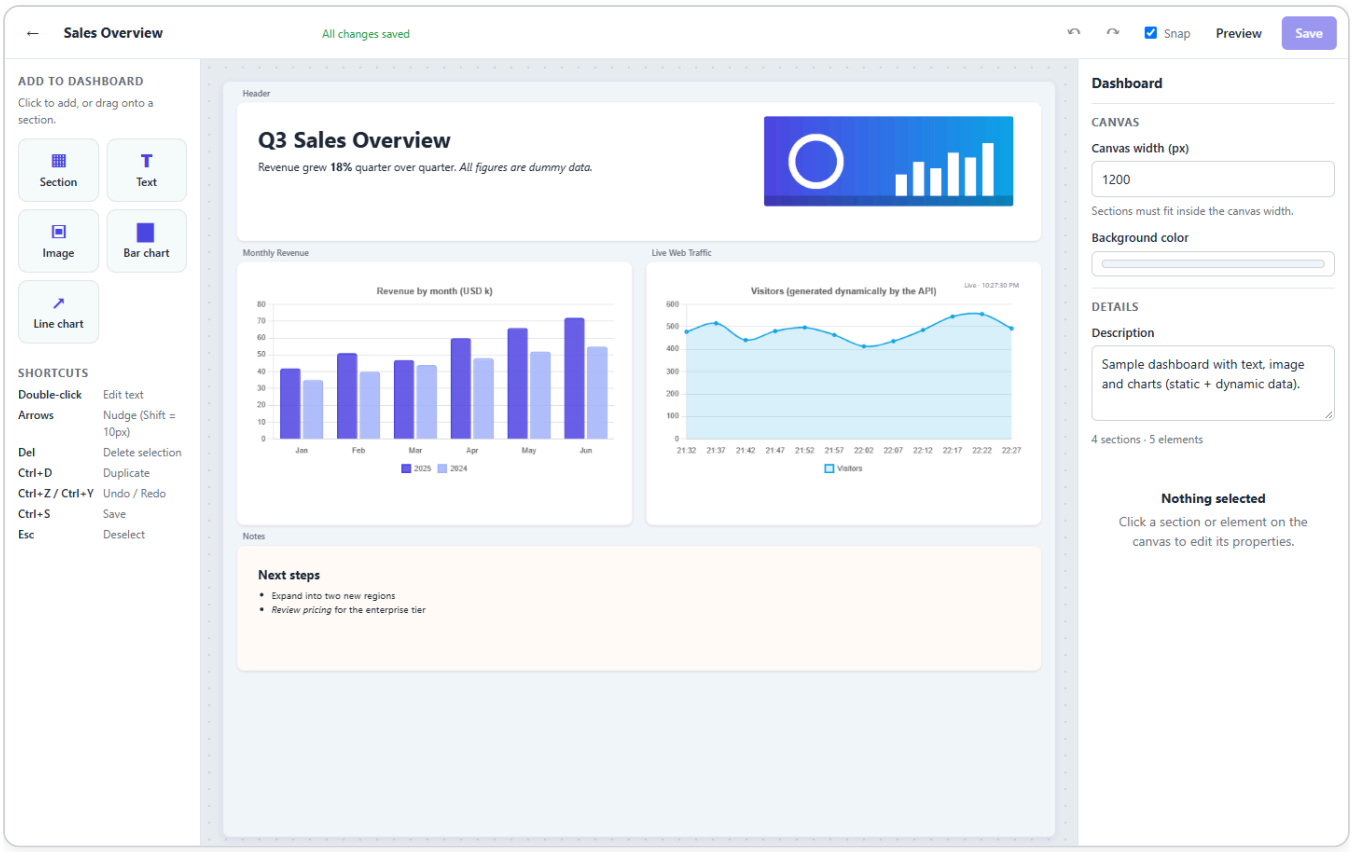
Part 5: How to use the app



1. Sign in page: use the demo account or create a new one



2. "My dashboards": open one, or click **+ New dashboard**



3. The builder: elements on the left, canvas in the middle, properties on the right

What you want to do	How
Add a section	Click **Section** in the left panel (or drag it onto the canvas)
Add text, image or chart	Click a section to select it, then click **Text**, **Image**, **Bar chart** or **Line chart**. You can also drag the item onto a section.
Move something	Click it and drag it with the mouse
Resize something	Click it, then drag one of the small white squares on its edges
Write and format text	**Double-click** a text box. Use the toolbar at the top for **bold**, *italic*, size and color. Press **Esc** when done.
Upload an image	Double-click an image box, or use **Upload image** on the right (PNG, JPG, GIF or WEBP, up to 5 MB)
Change chart data	Click the chart and edit the table on the right. Choose **Dynamic** as the data source for data that updates automatically.
Exact position or size	Type the numbers in X / Y / Width / Height on the right
Delete	Select it and press **Delete**, or use the red button on the right
Undo / Redo	**Ctrl + Z** / **Ctrl + Y** (Mac: Cmd instead of Ctrl)

Save	Click **Save** (top right) or press **Ctrl + S**. "All changes saved" means it's stored in the database.
See the final result	Click **Preview**. Click **Exit preview** to go back to editing.

> **Success check:** Reload the page after saving: everything comes back exactly where you left it, because it is loaded from the database.

Part 6: Stop, start again, reset

Task	Way A (Docker)	Way B (Node.js)
Start	`docker compose up`	`docker compose up -d mysql` then `npm run dev`
Stop	**Ctrl + C**, then `docker compose down`	**Ctrl + C**
Start fresh with the original sample data (**deletes your dashboards!**)	`docker compose down -v` then `docker compose up --build`	`npm run db:reset`
After changing code	`docker compose up --build`	Nothing: it reloads automatically
Run the automated tests	(not needed)	`npm test` (the database must be running)

Part 7: Something went wrong?

Find the message you see in the left column.

Problem / message	Fix
`docker: command not found` or `Cannot connect to the Docker daemon` or `error during connect`	Docker Desktop is not installed or not open. Open it and wait for **"Engine running"**, then try again.
`npm: command not found` or `'node' is not recognized`	Node.js isn't installed (Part 1, step 2). After installing, **close and reopen** the terminal.
`no configuration file provided` or `Could not read package.json`	The terminal is in the wrong folder. Open it inside the project folder (Part 2).
`port is already allocated` or `address already in use` for **3306**	Another MySQL is already running on your computer. Stop it (Windows: press Win+R, type `services.msc`, find **MySQL80**, click **Stop**). Or, in `docker-compose.yml`, change `"3306:3306"` to `"3307:3306"` and set `DB_PORT=3307` in `backend/.env`.
Port **5000** already in use (common on **Mac**)	macOS uses port 5000 for "AirPlay Receiver". Turn it off: **System Settings → General → AirDrop & Handoff → AirPlay Receiver → Off**. Then start again.
Port **5173** already in use	The app is probably already running in another terminal. Close the other terminal, or press Ctrl + C there.
Website says **"Cannot reach the server"**	The backend isn't running. In Way B, use `npm run dev` (it starts both parts) and check that the database is running: `docker compose up -d mysql`.
`Could not connect to MySQL (ECONNREFUSED)`	The database isn't running or is still starting. Run `docker compose up -d mysql`, wait 20 seconds, try again.
`ER_ACCESS_DENIED_ERROR`	Wrong database password. Check `DB_USER` / `DB_PASSWORD` in `backend/.env` (Docker database: `root` / `rootpass`).
`ER_BAD_DB_ERROR` or `Table ... doesn't exist`	The tables were not created yet. Run `npm run db:setup`.
`Invalid environment configuration`	The `backend/.env` file is missing, or `JWT_SECRET` is too short. Redo Part 4, step 3.
Login says **"Invalid email or password"** for the demo account	The sample data is missing. Way B: `npm run db:setup`. Way A: restart with `docker compose up`.
Page looks broken or old after changes	Hard-refresh the browser: **Ctrl + Shift + R** (Mac: Cmd + Shift + R).

Docker build fails with a network or timeout error	Check your internet connection and run `docker compose up --build` again.

> **Tip:** Still stuck? Send the person who gave you this project a **screenshot of the terminal** showing the red error text.

Part 8: Quick command cheat sheet

Way A: Docker

FIRST TIME

```
docker compose up --build
```

NEXT TIMES

```
docker compose up
```

STOP

```
docker compose down
```

Open: **http://localhost:5000**

Way B: Node.js

FIRST TIME

```
docker compose up -d mysql
npm install
(create backend/.env, see Part 4)
npm run db:setup
npm run dev
```

NEXT TIMES

```
docker compose up -d mysql
npm run dev
```

Open: **http://localhost:5173**

Demo login: **demo@example.com** / **Demo@1234**

For technical details (API, database design, security, folder structure), see `README.md` in the project folder.